# DA6401W - Assignment 4

Submitted by: Sohang Chopra <DA25M622>

## Problem 1: Theoritical

Consider training a deep neural network using stochastic gradient descent (SGD) on a non-convex loss surface. Curriculum learning is implemented by first training on "easy" examples and gradually introducing harder examples.

Which of the following statements is **most theoretically accurate** ?

1. Curriculum learning guarantees convergence to a flatter minimum.
2. Curriculum learning modifies early gradient statistics, thereby influencing the optimization trajectory in parameter space.
3. Curriculum learning changes the global minimum of the objective function.
4. Curriculum learning eliminates the effect of random initialization.

### Solution 1

Correct option is *Curriculum learning modifies early gradient statistics, thereby influencing the optimization trajectory in parameter space*.

Selecting "easy" examples first biases data distribution - the early gradients are different, and pushes early parameters into a parameter space which ideally enables skipping poor local minima once harder examples are introduced.

# Problem 5 (Conceptual MCQ) Nesterov Accelerated Gradient

Momentum methods are commonly used to accelerate gradient-based optimization in deep learning. Nesterov Accelerated Gradient (NAG) modifies the classical momentum method.

Which of the following statements best describes the key idea behind **Nesterov momentum** ?

1. The gradient is computed at the current parameter location without using momentum.
2. The gradient is computed after performing a *look-ahead* step using the momentum term.
3. The learning rate is automatically adjusted based on the magnitude of gradients.
4. The optimization step ignores past gradients completely.
5. The algorithm guarantees convergence to the global optimum for non-convex problems.

### Solution 5

Correct option is *The gradient is computed after performing a look-ahead step using the momentum term.*

That is, parameters are moved in direction of previous momentum before calculating gradients, and make a correction if momentum is about to carry it too far.

## Problem 7

Consider a fully connected neural network trained for a classification task. During training, **dropout** is applied to a hidden layer with dropout probability $p$ .

1. Explain how dropout helps prevent overfitting in neural networks. In your answer, discuss why dropout can be interpreted as implicitly training an ensemble of subnetworks.
2. During training, neurons are randomly dropped with probability $p$ . However, during inference (testing), dropout is not applied.
    - Explain how the network output is adjusted during inference to account for dropout used during training.
    - Why is this scaling necessary?
3. Suppose the output of a neuron before dropout is $h$ . Let the dropout mask be $m \sim Bernoulli(1-p)$. The output after dropout during training is $\bar{h} = mh$. Compute $E[\bar{h}]$.
4. Dropout is often applied to fully connected layers but less frequently to convolutional layers in modern architectures. Provide two reasons why dropout may be less effective in convolutional layers.

### Solution 7

1. During training, Dropout randomly disables different weights each epoch via a mask. This forces all weights to learn properly and prevents model from being over-reliant on just a few weights during training. Due to randomization, we can view this as each combination of model weights as a sub-network that recieves gradient updates seperately. Since all weights are used during inference, so this can be interpreted as implicitly training an ensemble of sub-networks.
2. At inference time, each neuron's output is scaled by retention probability $1-p$. This is necessary to approximate averaging all possible sub-networks. Without scaling, output of layer would be significantly higher at inference than during training (when some weights were randomly disabled).
3. Since mask $m$ is drawn from $Bernoulli(1-p)$, its expected value is $1-p$. So:

$$E[\bar{h}] = E[m \cdot h] = h \cdot E[m] = h(1-p)$$

4. Reasons:
    - In convolution layers, adjacent pixels are highly correlated. If a neuron is disabled, its adjacent neurons can still infer the missing information. This defeats Dropout's purpose of forcing indepedence amongst neurons.

- Usually there are much more neurons in fully-connected layers at the end than in Convolution layers. As Dropout is proportional to number of neurons in layer, it makes sense to focus on the fully-connected layers.
- BatchNorm layer is commonly used in Convolution Neural Networks and has an inherent regularizing effect. A mixture of Dropout and BatchNorm can cause performance to worsen.

## Problem 10 (Numerical Question) Label Smoothing

A classification task has $k = 4$ classes. Label smoothening with parameter $\epsilon = 0.1$ is applied. If the true class is class 2, compute the smoothed target distribution.

### Solution 10

In one-hot encoding of label, true class 2 becomes $1 - \epsilon = 1 - 0.1 = 0.9$, and remaining classes become $\epsilon/(K - 1) = 0.1/3$. So smoothed one-hot encoded label vector is $0.1/3, 0.1/3, 0.9, 0.1/3$ (assuming 0-based class index).

## Problem 11: Numerical: One Iteration of Steepest Descent

Consider the quadratic objective function:

$$f(w_1, w_2) = 2w_1^2 + w_1 w_2 + w_2^2$$

The steepest descent update rule is:

$$w_{t+1} = w_t - \eta \nabla f(w_t)$$

Given:

$$w_0 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad \eta = 0.1$$

Compute the updated parameter vector $w_1$ after one iteration of steepest descent.

### Solution 11

$$\nabla f = \begin{bmatrix} w_1 + w_2 \\ w_1 + 0.5w_2 \end{bmatrix}$$

$$w_1 = \begin{bmatrix} 2 \\ -1 \end{bmatrix} - 0.1 \begin{bmatrix} 2 - 1 \\ 2 - 0.5 \end{bmatrix} = \begin{bmatrix} 1.9 \\ -1.15 \end{bmatrix}$$

## Problem 13: Numerical: One Iteration of ADMM for Lasso Problem

*Note (ADMM intuition):* The Alternating Direction Method of Multipliers (ADMM) is an optimization algorithm used to solve problems where the objective function can be split into multiple components with simple structure.

In ADMM, the constrained problem

$$\min_{w,z} f(w) + g(z), \quad \text{subject to} \quad w = z$$

is solved by performing *alternating updates* :

- A quadratic minimization step for $w$
- A proximal (soft-thresholding) step for $z$
- A dual variable update enforcing consistency between $w$ and $z$

For the Lasso problem, the $z$-update corresponds to applying the **soft-thresholding operator**, which promotes sparsity in the solution.

Consider the Lasso optimization problem:

$$\min_{w} \frac{1}{2} \left\| Aw - b \right\|_2^2 + \lambda \left\| z \right\|_1, \quad \text{subject to} \quad w = z$$

Using the ADMM formulation, the augmented Lagrangian is:

$$L_p(w, z, u) = \frac{1}{2} \left\| Aw - b \right\|_2^2 + \lambda \left\| z \right\|_1 + \frac{\rho}{2} \left\| w - z + u \right\|_2^2$$

where $u$ is the scaled dual variable.

Given:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Initial values:

$$w_0 = z_0 = u_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Parameters: $\lambda = 0.5, \quad \rho = 1$

Compute:

1. The updated $w_1$
2. The updated $z_1$ using the soft-thresholding operator
3. The updated dual variable $u_1$

## Solution 13

The ADMM $w$-update is

$$w_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z_k - u_k))$$

$$A^T A = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$$

$$A^T A + \rho I = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$$

$$(A^T A + \rho I)^{-1} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{5} \end{bmatrix}$$

$$A^T b = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$$

$$A^T b + \rho(z_0 - u_0) = \begin{bmatrix} 1 \\ 4 \end{bmatrix} \quad (\text{since } z_0 - u_0 = 0)$$

$$w_1 = (A^T A + \rho I)^{-1}(A^T b + \rho(z_0 - u_0)) = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$$

z-update (soft thresholding):

- z-update rule: $z_{k+1} = S_{\lambda/\rho}(w_{k+1} + u_k)$ *Threshold: $\lambda/\rho = 0.5$
- Input vector: $w_1 + u_0 = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$
- Soft-threshold rule: $S_\kappa(x) = \text{sign}(x)\max(|x| - \kappa, 0)$

Apply elementwise.

**First element**: $S_{0.5}(0.5) = \max(0.5 - 0.5, 0) = 0$

**Second element**: $S_{0.5}(0.8) = 0.8 - 0.5 = 0.3$

So

$$z_1 = \begin{bmatrix} 0 \\ 0.3 \end{bmatrix}$$

Dual Variable Update:

$$u_{k+1} = u_k + w_{k+1} - z_{k+1}$$

Substitute values:

$$u_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.3 \end{bmatrix}$$

$$u_1 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

Final Results:

$$w_1 = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$$

$$z_1 = \begin{bmatrix} 0 \\ 0.3 \end{bmatrix}$$

$$u_1 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$